

Theory of Computation Problem Sets 1-6

Problem Set 1

0.1 Book 2.18

Question.

- (a) Prove that if C is a context-free language and R is a regular language, then $C \cap R$ is context free.
- (b) Use part (a) to show that $A = \{w \in \{a, b, c\}^* \mid \#a(w) = \#b(w) = \#c(w)\}$ is not context free.

(a) If C is context free and R is regular, then $C \cap R$ is context free.

Let P be a PDA for C and let D be a DFA for R . Build a new PDA P' whose state set is the product of the state sets of P and D . On each input symbol, P' simulates one step of P and one step of D in parallel, while keeping the same stack behavior that P uses. The accept states of P' are the pairs where both simulated machines are accepting. Therefore

$$L(P') = L(P) \cap L(D) = C \cap R.$$

Hence $C \cap R$ is context free.

(b) $A = \{w \in \{a, b, c\}^ \mid \#a(w) = \#b(w) = \#c(w)\}$ is not context free.*

Assume A were context free. Since regular languages are closed under intersection with CFLs, the language

$$A \cap a^*b^*c^*$$

would also be context free. But

$$A \cap a^*b^*c^* = \{a^n b^n c^n \mid n \geq 0\},$$

which is not context free. This contradiction shows that A is not context free.

0.2 Book 2.26

Question.

Show that if G is a CFG in Chomsky normal form, then every derivation of a string $w \in L(G)$ with $|w| = n > 1$ uses exactly $2n - 1$ steps.

Let G be in Chomsky normal form and let $w \in L(G)$ with $|w| = n > 1$.

In CNF every internal derivation step replaces one variable by two variables, and every terminal step replaces one variable by one terminal. A parse tree for a length- n string therefore has:

- exactly n leaves, one for each terminal of w ,
- exactly $n - 1$ internal binary nodes.

Each internal node corresponds to one application of a rule of the form $A \rightarrow BC$, and each leaf corresponds to one application of a rule of the form $A \rightarrow a$.

Therefore every derivation uses

$$(n - 1) + n = 2n - 1$$

steps.

1. Book 1.32

Question.

Let B be the language of three-row binary additions written columnwise. Show that B is regular.

Let B be the language of three-row binary additions written columnwise. It is easier to work with B^R , because reversing the string makes the least significant bit appear first.

For B^R , a DFA only needs to remember the carry from the previous column. There are two possibilities: carry 0 or carry 1. On reading a column whose top, middle, and bottom bits are (x, y, z) , the automaton checks whether $x + y + \text{carry}$ has low bit z and updates the carry to the high bit. If a column violates binary addition, the DFA moves to a dead state. After the final column, the DFA accepts exactly when the final carry is consistent with the leftmost column.

Thus B^R is regular. By Problem 1.31, reversal preserves regularity, so B is regular.

2. Book 1.41

Question.

Show that the class of regular languages is closed under perfect shuffle.

Let A and B be regular over the same alphabet Σ , recognized by DFAs

$$M_A = (Q_A, \Sigma, \delta_A, q_A, F_A), \quad M_B = (Q_B, \Sigma, \delta_B, q_B, F_B).$$

We build a DFA for the perfect shuffle of A and B .

The new DFA keeps:

- the current state of M_A ,
- the current state of M_B ,
- one bit indicating whose turn it is.

So the state set is $Q_A \times Q_B \times \{A\text{-turn}, B\text{-turn}\}$.

If the machine is in A-turn and reads symbol a , it updates only the M_A component and switches to B-turn. If it is in B-turn, it updates only the M_B component and switches back to A-turn. The machine accepts exactly when the turn bit says the input length is even and both component states are accepting.

Therefore the perfect shuffle of two regular languages is regular.

3. Book 1.53

Question.

Let

$$\text{ADD} = \{x = y + z \mid x, y, z \text{ are binary integers and } x \text{ is the sum of } y \text{ and } z\}.$$

Show that ADD is not regular.

Let

$$\text{ADD} = \{x = y + z \mid x, y, z \text{ are binary integers and } x \text{ is the sum of } y \text{ and } z\}.$$

Assume ADD were regular. Intersect it with the regular language

$$R = 10^* = 1 + 1^*.$$

Then

$$\text{ADD} \cap R = \{10^n = 1 + 1^n \mid n \geq 1\},$$

because 10^n is the binary representation of 2^n and 1^n is the binary representation of $2^n - 1$, so indeed

$$1 + (2^n - 1) = 2^n.$$

Now erase the fixed symbols 1, +, and = by a homomorphism that sends them to ε and keeps 0 and 1. The resulting language is

$$\{0^n 1^n \mid n \geq 1\},$$

which is not regular. This contradicts closure of regular languages under intersection and homomorphism. Hence ADD is not regular.

4. Book 1.60

Question.

For each $k \geq 1$, let $C_k = \Sigma^* a \Sigma^{k-1}$ over $\Sigma = \{a, b\}$. Describe an NFA with $k + 1$ states that recognizes C_k .

For $k \geq 1$, let

$$C_k = \Sigma^* a \Sigma^{k-1}, \quad \Sigma = \{a, b\}.$$

An NFA with $k + 1$ states recognizes C_k :

- q_0 is the start state and loops on both a and b ,
- on reading an a , q_0 may nondeterministically move to q_1 ,
- for $1 \leq i < k$, state q_i goes to q_{i+1} on both a and b ,
- q_k is the only accept state.

Intuitively, the NFA guesses which a is exactly k places from the end, and then checks that exactly $k - 1$ symbols remain.

Formally,

$$M = (\{q_0, q_1, \dots, q_k\}, \{a, b\}, \delta, q_0, \{q_k\}),$$

where

- $\delta(q_0, a) = \{q_0, q_1\}$,
- $\delta(q_0, b) = \{q_0\}$,
- $\delta(q_i, a) = \delta(q_i, b) = \{q_{i+1}\}$ for $1 \leq i < k$,
- $\delta(q_k, a) = \delta(q_k, b) = \emptyset$.

5. Book 1.61

Question.

For the languages C_k from Problem 1.60, prove that no DFA recognizing C_k can have fewer than 2^k states.

We prove that every DFA for C_k needs at least 2^k states by Myhill-Nerode.

Consider all strings of length k over $\{a, b\}$. There are 2^k of them. I claim they are pairwise distinguishable with respect to C_k .

Let x and y be distinct strings in Σ^k . Let j be the first position from the left where they differ. Without loss of generality, suppose $x_j = a$ and $y_j = b$. Let

$$z = b^{j-1}.$$

Then in the string xz , the symbol that is exactly k places from the end is the j th symbol of x , namely a , so $xz \in C_k$. In yz , the symbol exactly k places from the end is the j th symbol of y , namely b , so $yz \notin C_k$.

Thus x and y are distinguishable. Therefore C_k has at least 2^k equivalence classes, and any DFA for C_k must have at least 2^k states.

6. Book 2.24

Question.

Let

$$E = \{a^i b^j \mid i \neq j \text{ and } 2i \neq j\}.$$

Show that E is a context-free language.

Let

$$E = \{a^i b^j \mid i \neq j \text{ and } 2i \neq j\}.$$

Write E as the union of three CFLs:

$$E = L_1 \cup L_2 \cup L_3,$$

where

$$L_1 = \{a^i b^j \mid j < i\},$$

$$L_2 = \{a^i b^j \mid i < j < 2i\},$$

$$L_3 = \{a^i b^j \mid j > 2i\}.$$

Each is context free.

For L_1 , use the grammar

$$S_1 \rightarrow aS_1b \mid aS_1 \mid a.$$

This generates more a 's than b 's.

For L_3 , use

$$S_3 \rightarrow aS_3bb \mid B, \quad B \rightarrow bB \mid b.$$

This generates $a^i b^{2i+r}$ with $r \geq 1$.

For L_2 , use

$$S_2 \rightarrow aS_2b \mid aTb, \quad T \rightarrow aTbb \mid abb.$$

The nonterminal T generates $a^n b^{2n}$ with $n \geq 1$, and the outer wrapping by S_2 adds the same positive number of a 's and b 's. Therefore strings in L_2 have the form

$$a^{m+n+1} b^{m+2n+1},$$

so $j - i = n \geq 1$ and $2i - j = m + 1 \geq 1$, hence $i < j < 2i$.

Since CFLs are closed under union, E is context free.

7. Book 2.27

Question.

The natural grammar for the if-then-else language is ambiguous.

- (a) Show that it is ambiguous.
- (b) Give a new unambiguous grammar for the same language.

(a) The grammar is ambiguous.

Consider

if condition then if condition then a:=1 else a:=1.

It has two parses:

- 1. the else is matched with the inner if,
- 2. the else is matched with the outer if.

Hence the grammar is ambiguous.

(b) An unambiguous grammar.

Use the standard matched/unmatched decomposition:

```

STMT -> MATCHED | UNMATCHED
MATCHED -> a:=1
MATCHED -> if condition then MATCHED else MATCHED
UNMATCHED -> if condition then STMT
UNMATCHED -> if condition then MATCHED else UNMATCHED

```

This grammar is unambiguous because every else is forced to match the nearest unmatched if.

8. Book 1.59

Question.

If a k -state DFA is synchronizable, prove that it has a synchronizing sequence of length at most k^3 . State whether the bound can be improved.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a synchronizable DFA with $|Q| = k$.

First show that for any two states $p, q \in Q$, there is a word sending both to the same state whose length is less than k^2 . Consider the pair automaton on ordered pairs of states. Its states are $(r, s) \in Q \times Q$. Since M is synchronizable, from every pair (p, q) there is a path to some diagonal state (h, h) . A shortest such path cannot repeat a pair state, so its length is at most $k^2 - 1$.

Now start with the whole set Q . Repeatedly choose two distinct current states and apply a word that merges them. Each such word decreases the number of possible current states by at least 1, and each has length at most $k^2 - 1$. After at most $k - 1$ rounds, all states have merged into one state.

Therefore there exists a synchronizing word of length at most

$$(k - 1)(k^2 - 1) < k^3.$$

So the required bound follows.

This can indeed be improved. Classical results give substantially better cubic bounds, and the famous Cerny conjecture predicts the sharp bound $(k - 1)^2$.

Problem Set 2

1. Book 2.32

Question.

Let

$$C = \{w \in \{1,2,3,4\}^* \mid \#_1(w) = \#_2(w) \text{ and } \#_3(w) = \#_4(w)\}.$$

Show that C is not context free.

Let

$$C = \{w \in \{1,2,3,4\}^* \mid \#_1(w) = \#_2(w) \text{ and } \#_3(w) = \#_4(w)\}.$$

Assume C is context free. Intersect it with the regular language

$$R = 1^*3^*2^*4^*.$$

Then

$$C \cap R = \{1^m 3^n 2^m 4^n \mid m, n \geq 0\}.$$

I claim this language is not context free. Let p be a pumping length and take

$$s = 1^p 3^p 2^p 4^p.$$

For any decomposition $s = uvxyz$ with $|vxy| \leq p$ and $|vy| > 0$, the substring vxy lies within at most two adjacent blocks. Pumping v and y changes at most two of the four block counts, so after pumping the equality between the number of 1's and 2's or the equality between the number of 3's and 4's is destroyed. Hence the pumped string leaves the language, contradicting the CFL pumping lemma.

Therefore $C \cap R$ is not context free, so C itself is not context free.

2. Book 3.12

Question.

Show that Turing machines with left reset recognize exactly the Turing-recognizable languages.

We show that left-reset Turing machines recognize exactly the Turing-recognizable languages.

One direction is easy: an ordinary TM can simulate RESET by repeatedly moving left until it reaches the left endmarker, so every left-reset TM is Turing-recognizable in the ordinary model.

For the converse, simulate an ordinary TM M by a left-reset TM R . On its tape, R stores the simulated tape contents together with a marker indicating the simulated head position. To simulate a right move, R changes the marked symbol, performs RESET, scans right from the left end until it finds the marked cell, and moves the marker one position to the right.

To simulate a left move, R again resets to the left end and scans right until it finds the marked cell; during this scan it remembers the immediately preceding cell in finite control. It then performs another pass to move the marker onto that previous cell. Since every simulated left move can be carried out by a bounded number of reset-and-scan passes, R can simulate every step of M .

Hence left-reset TMs recognize exactly the Turing-recognizable languages.

3. Book 3.18

Question.

Prove that a language is decidable if and only if it is enumerable in lexicographic order.

We prove both directions.

If A is decidable, then enumerate all strings in lexicographic order and run the decider on each one. Print exactly the strings it accepts. This enumerates A in lexicographic order.

Conversely, suppose A is enumerable in lexicographic order by some enumerator E . To decide whether a given string w is in A , run E until one of the following happens:

- E prints w , in which case accept;
- E prints some string lexicographically larger than w , in which case reject.

Because E prints strings in increasing lexicographic order, if w were going to appear, it would appear before any larger string. Hence this procedure halts and is correct.

Therefore a language is decidable iff it is enumerable in lexicographic order.

4. Book 4.17

Question.

Let C be a language. Prove that C is Turing-recognizable if and only if there exists a decidable language D such that

$$C = \{x \mid \exists y ((x, y) \in D)\}.$$

If C is Turing-recognizable, then such a decidable D exists.

Let M recognize C . Define

$$D = \{(x, y) \mid y \text{ encodes an accepting computation history of } M \text{ on input } x\}.$$

Checking whether y is a valid accepting computation history is decidable: verify that the first configuration is the start configuration on x , each successive configuration legally follows from the previous one, and the last configuration is accepting.

Now $x \in C$ iff there exists some accepting computation history y for M on x , i.e.

$$x \in C \Leftrightarrow \exists y ((x, y) \in D).$$

If such a decidable D exists, then C is Turing-recognizable.

On input x , systematically enumerate all strings y and run the decider for D on (x, y) . If any test accepts, accept x . This machine recognizes C because

$$x \in C \Leftrightarrow \exists y ((x, y) \in D).$$

So C is Turing-recognizable iff it is the projection of some decidable language.

5. Book 4.22

Question.

Formulate the problem of determining whether a PDA has a useless state as a language, and prove that it is decidable.

Let

$$\text{USELESSPDA} = \{\langle P \rangle \mid P \text{ is a PDA having a useless state}\}.$$

We show USELESSPDA is decidable.

Fix a PDA P and a state q of P . Construct a new PDA P_q that simulates P exactly, except that the first time it enters q it may jump by an ε -move to a fresh accepting state and halt. Then

$$L(P_q) \neq \emptyset$$

iff state q is reachable on some input string in some computation of P .

Emptiness for PDAs is decidable, so for each state q we can decide whether $L(P_q)$ is empty. If some state yields an empty language, that state is useless; otherwise none is.

Since P has only finitely many states, this procedure halts. Therefore USELESSPDA is decidable.

6. Book 4.25

Question.

Let

$$E = \{\langle M \rangle \mid M \text{ is a DFA that accepts some string with more 1s than 0s}\}.$$

Show that E is decidable.

Let

$$E = \{\langle M \rangle \mid M \text{ is a DFA that accepts some string with more 1s than 0s}\}.$$

Consider the language

$$B = \{w \in \{0,1\}^* \mid \#_1(w) > \#_0(w)\}.$$

B is a deterministic CFL. A PDA for B keeps on its stack the unmatched majority symbol:

- on input 1, if the stack top is $\emptyset$, pop; otherwise push 1,
- on input $\emptyset$, if the stack top is 1, pop; otherwise push $\emptyset$.

After all cancellations, the stack contains only copies of one symbol. Accept iff at least one unmatched 1 remains.

Now, given DFA M , the language $L(M) \cap B$ is context free because CFLs are closed under intersection with regular languages. Build a PDA for that intersection and test emptiness, which is decidable for PDAs/CFGs.

Then

$$\langle M \rangle \in E \Leftrightarrow L(M) \cap B \neq \emptyset.$$

Hence E is decidable.

7. Book 2.22

Question.

Let

$$C = \{x\#y \mid x, y \in \{0,1\}^* \text{ and } x \neq y\}.$$

Show that C is a context-free language.

Let

$$C = \{x\#y \mid x, y \in \{0,1\}^* \text{ and } x \neq y\}.$$

Split C into two CFLs:

$$C = C_{\text{len}} \cup C_{\text{diff}},$$

where

- C_{len} contains all strings $x\#y$ with $|x| \neq |y|$,
- C_{diff} contains all strings $x\#y$ with $|x| = |y|$ but $x \neq y$.

C_{len} is context free: a PDA can compare the two lengths by pushing for symbols before $\#$ and popping after $\#$, accepting if the stack empties too early or has leftovers at the end.

C_{diff} is also context free: the PDA pushes the symbols of x ; after $\#$ it nondeterministically chooses one position at which to witness a mismatch, remembers the expected symbol in the state, and otherwise simply matches and pops. It accepts exactly if the chosen position differs and all lengths match.

Since CFLs are closed under union, C is context free.

Problem Set 3

1. Book 5.22 and 5.23

Question.

Prove both statements:

- A is Turing-recognizable iff $A \leq_m A_{TM}$.
- A is decidable iff $A \leq_m 0^*1^*$.

5.22: A is Turing-recognizable iff $A \leq_m A_{TM}$.

If A is recognized by TM M , define

$$f(w) = \langle M, w \rangle.$$

Then

$$w \in A \Leftrightarrow \langle M, w \rangle \in A_{TM}.$$

So $A \leq_m A_{TM}$.

Conversely, if $A \leq_m A_{TM}$ and A_{TM} is Turing-recognizable, then Theorem 5.28 implies that A is Turing-recognizable.

*5.23: A is decidable iff $A \leq_m 0^*1^*$.*

If A is decidable by decider M , define $f(w)$ to be 0 if M accepts w , and 10 if M rejects w .

Since $0 \in 0^*1^*$ and $10 \notin 0^*1^*$, we get

$$w \in A \Leftrightarrow f(w) \in 0^*1^*.$$

So $A \leq_m 0^*1^*$.

Conversely, if $A \leq_m 0^*1^*$, then because 0^*1^* is decidable, Theorem 5.22 gives that A is decidable.

2. Book 5.25

Question.

Give an example of an undecidable language B such that $B \leq_m B^c$. Also determine whether such a B can be Turing-recognizable.

Take

$$B = \{0x \mid x \in A_{TM}\} \cup \{1x \mid x \notin A_{TM}\}.$$

B is undecidable because deciding B would decide A_{TM} : to test whether $x \in A_{TM}$, simply test whether $0x \in B$.

Now define

$$f(0x) = 1x, \quad f(1x) = 0x.$$

Then f is computable and

$$w \in B \Leftrightarrow f(w) \notin B.$$

Equivalently,

$$w \in B \Leftrightarrow f(w) \in B^c.$$

So $B \leq_m B^c$.

Can B be Turing-recognizable? No. If B were recognizable, then so would be B^c by Theorem 5.28, because $B \leq_m B^c$ and also $B^c \leq_m B$ via the same bit-flip map. Then Theorem 4.22 would imply that B is decidable, contradiction. Hence this B is not Turing-recognizable.

3. Book 5.26

Question.

For two-headed finite automata (2DFAs):

- (a) Show that A_{2DFA} is decidable.
- (b) Show that E_{2DFA} is undecidable.

(a) A_{2DFA} is decidable.

A configuration of a 2DFA on input x is determined by:

- the current state,
- the position of head 1,
- the position of head 2.

If $|x| = n$, each head has only $n + 2$ possible positions (including endmarkers), so the total number of configurations is

$$|Q|(n + 2)^2.$$

Simulate the 2DFA step by step. If it enters an accept state, accept. If a configuration repeats, the machine is looping forever and will never accept, so reject. Because the number of configurations is finite, this always halts.

Hence A_{2DFA} is decidable.

(b) E_{2DFA} is undecidable.

Reduce A_{TM} to $(E_{2DFA})^c$. Given $\langle M, w \rangle$, construct a 2DFA $N_{M,w}$ that accepts exactly the valid accepting computation histories of M on input w , written as

$$C_1 \# C_2 \# \dots \# C_t.$$

Two heads let $N_{M,w}$ compare adjacent configurations symbol-by-symbol and verify local consistency, while finite control checks that:

- C_1 is the start configuration of M on w ,
- each C_{i+1} legally follows C_i ,
- the last configuration is accepting.

Thus

$$L(N_{M,w}) \neq \emptyset \Leftrightarrow M \text{ accepts } w.$$

So

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow \langle N_{M,w} \rangle \in (E_{2DFA})^c.$$

If E_{2DFA} were decidable, then so would be its complement, which would decide A_{TM} , impossible. Therefore E_{2DFA} is undecidable.

4. Book 5.21

Question.

Let $AMBIGCFG = \{\langle G \rangle \mid G \text{ is an ambiguous CFG}\}$. Show that $AMBIGCFG$ is undecidable.

We reduce PCP to AMBIGCFG.

Given a PCP instance with dominoes $(t_1, b_1), \dots, (t_k, b_k)$,

construct the CFG from the hint:

$$S \rightarrow T|B$$

$$T \rightarrow t_1 T a_1 | \dots | t_k T a_k | t_1 a_1 | \dots | t_k a_k$$

$$B \rightarrow b_1 B a_1 | \dots | b_k B a_k | b_1 a_1 | \dots | b_k a_k,$$

where $a_1, \dots, a_k$ are fresh terminal symbols.

Any derivation through T generates a string of the form

$$t_{i_1}t_{i_2}\cdots t_{i_m}a_{i_m}\cdots a_{i_2}a_{i_1},$$

and any derivation through B generates

$$b_{i_1}b_{i_2}\cdots b_{i_m}a_{i_m}\cdots a_{i_2}a_{i_1}.$$

Therefore a string has two distinct derivations iff there is a sequence $i_1\cdots i_m$ with

$$t_{i_1}\cdots t_{i_m} = b_{i_1}\cdots b_{i_m},$$

which is exactly a PCP match.

Hence the PCP instance has a solution iff G is ambiguous. Since PCP is undecidable, AMBIGCFG is undecidable.

5. Book 6.1

Question.

Give an example, in the spirit of the recursion theorem, of a program that prints its own source code.

A standard quine in Python is:

```
s = "s = %r\nprint(s % s)"
print(s % s)
```

When the program runs, the string s contains the source code template, and `print(s % s)` substitutes the literal representation of s back into the template. The printed output is exactly the program text itself, so this is a concrete example of the recursion theorem phenomenon.

6. Book 6.11

Question.

Give a model of the sentence φ_{lt} .

A model of φ_{lt} is

$$\mathcal{M} = (\mathbb{N}, =, <).$$

Interpret:

- $R_1(x, y)$ as $x = y$,
- $R_2(x, y)$ as $x < y$.

Then:

- R_1 satisfies reflexivity, symmetry in the encoded sense, and transitivity,

- equality and strict order are disjoint,
- if $x \neq y$, then exactly one of $x < y$ or $y < x$ holds,
- $<$ is transitive,
- every two elements are comparable by $=$ or $<$ or its reverse.

Thus $(\mathbb{N}, =, <)$ satisfies the sentence.

7. Book 4.14

Question.

Show that the problem of testing whether a CFG generates exactly 1^* is decidable.

We must decide whether a CFG G satisfies

$$L(G) = 1^*.$$

Break the question into two parts.

Step 1: Check $L(G) \subseteq 1^$.*

Intersect $L(G)$ with the regular language

$$\Sigma^* \setminus 1^*,$$

and test emptiness. This is decidable because CFLs are closed under intersection with regular languages and emptiness for CFGs is decidable.

Step 2: Check $1^ \subseteq L(G)$.*

Form a grammar G_1 for

$$L(G) \cap 1^*.$$

This is a unary context-free language. By Parikh's theorem in one dimension, the set

$$S = \{n \mid 1^n \in L(G)\}$$

is effectively semilinear, hence effectively a finite union of arithmetic progressions and finite exceptional sets. Therefore membership in S is ultimately periodic, and from an effective semilinear description we can decide whether $S = \mathbb{N}$.

So we can decide whether every 1^n belongs to $L(G)$.

Both checks are decidable, hence the problem is decidable.

Problem Set 4

1. Book 7.12

Question.

Show that

$$\text{MODEXP} = \{(a, b, c, p) \mid a^b \equiv c \pmod{p}\}$$

belongs to P .

We must decide whether

$$a^b \equiv c \pmod{p}$$

for binary integers a, b, c, p .

Use repeated squaring. Write b in binary as

$$b_m b_{m-1} \cdots b_1 b_0.$$

Compute the sequence

$$a^{2^0} \bmod p, a^{2^1} \bmod p, a^{2^2} \bmod p, \dots$$

by repeatedly squaring modulo p . Then multiply together exactly those powers for which $b_i = 1$, always reducing modulo p after each multiplication.

This uses $O(\log b)$ modular squarings and multiplications. Since arithmetic on numbers of size polynomial in the input length is polynomial time, the whole algorithm runs in polynomial time.

Therefore MODEXP is in P .

2. Book 7.14

Question.

Show that P is closed under the star operation.

Let $A \in P$, decided by algorithm M in time $p(n)$.

To decide whether a string $y = y_1 \cdots y_n$ lies in A^* , use dynamic programming. Define a table $T[0], \dots, T[n]$ by:

- $T[0] = \text{TRUE}$,
- for $i > 0$,

$$T[i] = \bigvee_{0 \leq j < i} (T[j] \wedge (y_{j+1} \cdots y_i \in A)).$$

Intuitively, $T[i]$ says whether the prefix of length i can be split into blocks from A .

There are $O(n^2)$ substrings $y_{j+1} \cdots y_i$, and each membership test is polynomial by running M . So the total time is polynomial.

Hence $A^* \in P$, and therefore P is closed under star.

3. Book 7.17

Question.

Assume $P = NP$. Show that every nontrivial language in P is NP-complete.

Assume $P = NP$. Let $A \in P$ with $A \neq \emptyset$ and $A \neq \Sigma^*$.

To show A is NP-complete, let B be any language in NP. By the assumption, $B \in P$, so membership in B is decidable in polynomial time.

Because A is nontrivial, pick fixed strings

- $x_{\text{yes}} \in A$,
- $x_{\text{no}} \notin A$.

Define $f(w)$ to be x_{yes} if $w \in B$, and x_{no} otherwise.

Since $B \in P$, f is computable in polynomial time, and

$$w \in B \Leftrightarrow f(w) \in A.$$

Thus $B \leq_p A$. Since B was arbitrary in NP, every NP language reduces to A . Also $A \in P \subseteq NP$.

Therefore A is NP-complete.

4. Book 7.26

Question.

Let *PUZZLE* be the card-and-box covering problem from the text. Show that *PUZZLE* is NP-complete.

We show *PUZZLE* is NP-complete.

PUZZLE is in NP.

A certificate is simply the orientation chosen for each card. Given these orientations, we can check in polynomial time whether every hole position in the box is blocked by at least one card. So *PUZZLE* is in NP.

NP-hardness.

Reduce from 3SAT. For a formula

$$\varphi = C_1 \wedge \cdots \wedge C_m,$$

create one hole position for each clause. For each variable x , create a card whose two possible orientations represent setting $x = \text{TRUE}$ or $x = \text{FALSE}$.

Design the card so that:

- in the TRUE orientation, the clause positions where x appears positively are blocked and the clause positions where $\neg x$ appears are left open;
- in the FALSE orientation, the opposite happens.

All other clause positions on that card are irrelevant and can be made holes on both sides.

Now the entire stack blocks clause position j iff at least one variable card is oriented in a way that makes some literal of clause C_j true. Therefore all hole positions are blocked iff every clause has a true literal, i.e. iff φ is satisfiable.

The construction is polynomial, so $3\text{SAT} \leq_p \text{PUZZLE}$. Hence PUZZLE is NP-complete.

5. Book 7.32

Question.

Let

$$U = \{(M, x, \#^t) \mid M \text{ accepts } x \text{ within } t \text{ steps on at least one branch}\}.$$

Show that U is NP-complete.

Let

$$U = \{(M, x, \#^t) \mid M \text{ accepts } x \text{ within } t \text{ steps on some branch}\}.$$

$U \in \text{NP}$.

A certificate is an accepting computation branch of length at most t . We can verify in polynomial time that:

- the branch starts in the correct start configuration,
- each step legally follows from the previous one,
- the final configuration is accepting,
- the branch length is at most t .

So $U \in NP$.

U is NP-hard.

Let $A \in NP$ and let N be a nondeterministic polynomial-time TM deciding A in time $p(n)$. Map input w to

$$f(w) = (N, w, \#^{p(|w|)}).$$

Then

$$w \in A \Leftrightarrow N \text{ has an accepting branch on } w \text{ within } p(|w|) \text{ steps} \Leftrightarrow f(w) \in U.$$

The map is polynomial-time computable, so every language in NP reduces to U .

Therefore U is NP-complete.

6. Book 7.37

Question.

Assume $P = NP$. Prove that integers can then be factored in polynomial time.

Assume $P = NP$.

Define

$$\text{FACTOR} = \{(n, \ell, r) \mid n \text{ has a nontrivial factor } d \text{ with } \ell \leq d \leq r\}.$$

FACTOR is in NP because a certificate is the factor d . Hence, under $P = NP$, FACTOR is in P .

Now to factor n , do a binary search on the interval $[2, n - 1]$ using the FACTOR decision procedure. At each stage ask whether a factor exists in the left half of the current interval; if yes, recurse there, otherwise recurse in the right half. This finds a nontrivial factor d in polynomially many oracle calls, hence in polynomial time.

Apply the same procedure recursively to d and n/d until all factors found are prime. Thus integer factorization is polynomial-time computable if $P = NP$.

7. Book 7.49

Question.

For resolution on CNF formulas:

- (a) Show that resolution is sound and complete.
- (b) Use this to prove that $2SAT \in P$.

(a) Resolution is sound and complete.

Soundness: if $C_a = (x \vee Y)$ and $C_b = (\neg x \vee Z)$, then any assignment satisfying both parents must satisfy $Y \vee Z$. So the resolvent is a logical consequence of the original clauses. Therefore resolution can never derive a false claim of unsatisfiability.

Completeness: the standard resolution theorem says that every unsatisfiable CNF formula has a refutation deriving the empty clause. Equivalently, if the closure of the clause set under resolution does not contain the empty clause, the clauses are simultaneously satisfiable. Hence resolution is complete.

(b) 2SAT $\in P$.

If every clause has at most two literals, every resolvent also has at most two literals. Over n variables there are only:

- $2n$ one-literal clauses,
- $O(n^2)$ two-literal clauses.

So the total number of distinct possible clauses is polynomial. We can repeatedly add all possible resolvents until closure is reached in polynomial time, and then check whether the empty clause was derived.

By part (a), the formula is unsatisfiable iff the empty clause appears. Therefore 2SAT is decidable in polynomial time, so $2SAT \in P$.

Problem Set 5

1. Book 7.24

Question.

For NAE-satisfiability:

- Show that the negation of any NAE-assignment is also an NAE-assignment.
- Give the reduction from 3SAT to NAE-3SAT.
- Conclude that NAE-3SAT is NP-complete.

(a) The negation of any NAE-assignment is also an NAE-assignment.

If a clause has at least one true literal and at least one false literal under an assignment α , then after flipping every truth value, the formerly true literals become false and the formerly false literals become true. The clause still has at

least one true and at least one false literal. So the negated assignment is again an NAE-assignment.

(b) Reduction from 3SAT to NAE-3SAT.

Replace each clause

$$c_i = (y_1 \vee y_2 \vee y_3)$$

by

$$(y_1 \vee y_2 \vee z_i) \wedge (\neg z_i \vee y_3 \vee b),$$

where each z_i is new and b is one new variable shared by all clauses.

If the original clause is satisfiable, choose b arbitrarily and then choose z_i so that each of the two new clauses has both a true and a false literal. Conversely, from any NAE-assignment to the new formula, at least one of y_1, y_2, y_3 must be true, for otherwise the two clauses would force contradictory values on z_i .

Thus the original formula is satisfiable iff the transformed formula has an NAE-assignment.

(c) Conclusion.

NAE-3SAT is in NP because an assignment is a polynomial certificate. By part (b), 3SAT reduces to it in polynomial time. Therefore NAE-3SAT is NP-complete.

2. Book 7.25

Question.

Show that *MAX-CUT* is NP-complete.

We show MAX-CUT is NP-complete.

MAX-CUT is in NP.

A certificate is the partition (S, T) . We can count the edges crossing the cut in polynomial time and check whether the number is at least k .

NP-hardness.

Reduce from NAE-3SAT. Use the standard gadget construction suggested in the problem.

- For each variable x , make a variable gadget consisting of many nodes labeled x and many nodes labeled $\neg x$, with all possible edges between the

- two groups. Any maximum cut places the x nodes on one side and the $\neg x$ nodes on the other, thereby encoding a truth value.
- For each clause, create a triangle on three fresh literal nodes labeled by the clause's literals. A triangle contributes at most 2 crossing edges, and it contributes exactly 2 iff not all three literal nodes lie on the same side of the cut. That is exactly the NAE condition.
 - Connect each clause-literal node strongly to the matching side of the corresponding variable gadget so that any cut of sufficiently large size must place the literal node consistently with the truth value selected for that variable.

Set the target k to be the sum of the forced contributions from the variable gadgets and consistency edges plus 2 per clause gadget.

Then the graph has a cut of size at least k iff the original formula has an NAE-satisfying assignment. Therefore $\text{NAE-3SAT} \leq_p \text{MAX-CUT}$, and MAX-CUT is NP-complete.

3. Book 8.15

Question.

Show that the cat-and-mouse game language *HAPPY-CAT* belongs to P .

Let a game state be a triple

$$(c, m, \tau),$$

where c is Cat's position, m is Mouse's position, and τ indicates whose turn it is. There are only $O(|V|^2)$ such states.

Classify states as follows:

- Cat-win immediately if $c = m$,
- Mouse-win immediately if $m = h$ and $c \neq m$.

Now work backward over the state graph, exactly as in finite game solving:

- a Cat-turn state is Cat-winning if it has a move to a Cat-winning state;
- a Mouse-turn state is Cat-winning if all moves lead to Cat-winning states;
- symmetrically define Mouse-winning states.

Iterate these rules until no new states are labeled. Any remaining unlabeled states are draws.

Because the configuration graph has polynomial size, this retrograde analysis runs in polynomial time. Therefore HAPPY-CAT is in P .

4. Book 8.10

Question.

Show that generalized go-moku belongs to $PSPACE$.

On an $n \times n$ board there are at most n^2 moves, because each move fills one empty square and no square is ever reused.

A depth-first minimax search decides whether player X has a winning strategy:

- if the current position is already winning for X or O , return that value;
- otherwise recursively evaluate all legal next moves and apply the usual “exists/all” rule depending on whose turn it is.

At any point we need to store only:

- the current board position,
- whose turn it is,
- the recursion stack.

The recursion depth is at most n^2 , and each board uses polynomial space, so the total space is polynomial. Therefore generalized go-moku is in $PSPACE$.

5. Book 8.8

Question.

Show that

$$EQREX = \{(R, S) | R \text{ and } S \text{ are equivalent regular expressions}\}$$

belongs to $PSPACE$.

Let

$$EQREX = \{(R, S) | R \text{ and } S \text{ are equivalent regular expressions}\}.$$

Convert R and S to equivalent NFAs N_R and N_S of polynomial size. The expressions are inequivalent iff there exists a string w accepted by exactly one of the two languages.

Determinize on the fly: a state of the subset construction for N_R is a subset of states of N_R , and similarly for N_S . A configuration of the comparison procedure is therefore a pair (X, Y) of subsets. The initial pair is the pair of ε -closures of the

start states. A pair is accepting-for-difference iff exactly one of X, Y contains an accepting NFA state.

So we only need to know whether, in the exponential-size product graph of subset states, an accepting-for-difference pair is reachable. Reachability in an exponential graph can be decided in polynomial space using Savitch's theorem.

Hence inequivalence is in PSPACE, and therefore equivalence is also in PSPACE. So $EQREX \in PSPACE$.

6. Book 8.13

Question.

Show that

$$ALBA = \{(M, w) \mid M \text{ is an LBA that accepts } w\}$$

is PSPACE-complete.

We show ALBA is PSPACE-complete.

ALBA is in PSPACE.

If an LBA M runs on input w , it uses only the cells occupied by w , so the number of possible configurations is exponential in $|w|$. A deterministic TM can do a depth-first search of the configuration graph using polynomial space, accepting iff it finds an accepting configuration reachable from the start configuration.

Thus ALBA is in PSPACE.

ALBA is PSPACE-hard.

Let A be any language in PSPACE, decided by TM N using at most $p(n)$ tape cells. On input x , map x to

$$\langle M, x\#^{p(|x|)-|x|} \rangle,$$

where M is an LBA that interprets the padded string as the original input x plus enough blank work space and then simulates N within that bounded tape region.

Then

$$x \in A \Leftrightarrow \langle M, x\#^{p(|x|)-|x|} \rangle \in ALBA.$$

This reduction is polynomial. Therefore ALBA is PSPACE-hard, and so ALBA is PSPACE-complete.

7. Book 7.35

Question.

You are given a fixed state set Q , a start state q_0 , and sample constraints of the form $\delta(q_0, s_i) = r_i$. Show that determining whether some DFA satisfying all constraints exists is NP-complete.

We show the DFA-construction problem is NP-complete.

Membership in NP.

A certificate is the transition table of the DFA. Given the table, we can simulate the DFA on each sample string s_i and verify in polynomial time that $\delta(q_0, s_i) = r_i$ for all i .

NP-hardness.

We reduce from 3SAT. The idea is to encode each variable by a partially specified behavior of the automaton from a shared prefix state. From that state, the two transitions on 0 and 1 represent the two truth values. Additional sample strings force all occurrences of the same variable to use the same choice, and clause strings are arranged so that a requested target state is reachable iff at least one literal in the clause is made true by those choices.

Concretely, the samples are built over a prefix tree:

- one family of strings forces a consistent binary choice for each variable,
- another family forces clause gadgets to reach a distinguished satisfied state whenever one of their literals is chosen consistently,
- a final family prevents unsatisfied clauses from reaching that state.

Thus a DFA meeting all sample constraints exists iff the original formula is satisfiable.

This is the standard “consistent transition assignment” reduction, so the problem is NP-hard and therefore NP-complete.

Problem Set 6

1. Book 8.22

Question.

Prove both:

- $ADD \in L$.
- $PAL-ADD \in L$.

(a) $ADD \in L$.

To test whether binary integers x, y, z satisfy $x + y = z$, scan from right to left with a carry bit. At step i , read the i th bits of x, y, z (using repeated scans from the left if needed), check whether

$$x_i + y_i + \text{carry}$$

has low bit z_i , and update the carry to the high bit.

Only the following information must be stored:

- a counter for the current bit position,
- the carry bit,
- a few position markers.

All of this uses $O(\log n)$ space. Hence $ADD \in L$.

(b) $PAL-ADD \in L$.

First determine the binary length of $x + y$ by running the logspace addition procedure and checking whether a final carry appears.

Then for each position i from the left, recompute on demand:

- the i th bit of $x + y$,
- the symmetric bit from the right.

Compare them. If all mirrored pairs match, the sum is a palindrome.

Because each bit can be recomputed in logspace and only $O(\log n)$ counters are needed, the whole algorithm uses logarithmic space. Therefore $PAL-ADD \in L$.

2. Book 8.27

Question.

Show that *STRONGLY-CONNECTED* is NL-complete.

We show *STRONGLY-CONNECTED* is NL-complete.

STRONGLY-CONNECTED is in NL.

A directed graph G is strongly connected iff for some fixed node s , every node is reachable from s and s is reachable from every node. *PATH* is in NL, and by the theorem $NL = coNL$, nonreachability is also in NL. Therefore checking the

absence of a violating node can be done in NL, so STRONGLY-CONNECTED is in NL.

NL-hardness.

Reduce PATH to STRONGLY-CONNECTED. Given (G, s, t) , construct H by:

- keeping all edges of G ,
- adding an edge from every vertex $v \neq t$ to s ,
- adding an edge from t to every vertex.

If s reaches t in G , then in H every vertex reaches s , then s reaches t , and from t every vertex is reachable, so H is strongly connected.

If s does not reach t in G , then no new edge creates a path from s to t because the only newly added incoming edges to t come from t itself via the original graph structure. So H is not strongly connected.

The construction is logspace computable, hence STRONGLY-CONNECTED is NL-complete.

3. Book 9.19

Question.

Let

$$USAT = \{(\varphi) \mid \varphi \text{ has exactly one satisfying assignment}\}.$$

Show that $USAT \in P^{SAT}$.

We show $USAT$ is in P^{SAT} .

Given a Boolean formula φ :

1. Query SAT on φ . If unsatisfiable, reject.
2. For each variable x_i , query SAT on $\varphi \wedge x_i$ and on $\varphi \wedge \neg x_i$.

If for some variable both restricted formulas are satisfiable, then there are two satisfying assignments differing on x_i , so reject.

If for every variable exactly one of the two restrictions is satisfiable, then the value of every variable is uniquely forced, hence there is exactly one satisfying assignment, so accept.

This uses only polynomially many SAT queries, therefore $USAT \in P^{SAT}$.

4. Book 9.17

Question.

Show that P contains a language that is not recognizable by any two-headed finite automaton.

From Problem 5.26(a), the acceptance problem for 2DFAs is decidable by brute-force simulation over all configurations. In fact, on input $\langle M, x \rangle$ the simulation runs in polynomial time because there are only $|Q|(|x| + 2)^2$ configurations.

Now diagonalize. Define

$$D = \{\langle M \rangle \mid M \text{ is a 2DFA and } M \text{ does not accept } \langle M \rangle\}.$$

To decide D , simulate the polynomial-time decider for A_{2DFA} on $\langle M, \langle M \rangle \rangle$ and invert the answer. Hence $D \in P$.

But no 2DFA can recognize D , because if some 2DFA N recognized D , then asking whether $\langle N \rangle \in D$ yields the contradiction

$$\langle N \rangle \in D \Leftrightarrow N \text{ does not accept } \langle N \rangle.$$

Therefore P contains a language not recognizable by any 2DFA.

5. Book 10.21

Question.

Show that

$$EQBP = \{(B_1, B_2) \mid B_1 \text{ and } B_2 \text{ are equivalent branching programs}\}$$

is coNP-complete.

We show EQBP is coNP-complete.

EQBP is in coNP.

Its complement consists of pairs of branching programs (B_1, B_2) for which there exists an assignment a such that $B_1(a) \neq B_2(a)$. That assignment is a polynomial-size certificate, and we can evaluate both branching programs on it in polynomial time. Hence $EQBP^c \in NP$, so $EQBP \in coNP$.

coNP-hardness.

Reduce SAT to $EQBP^c$. Given a CNF formula φ , build a branching program B_φ that reads the variables clause by clause and accepts exactly those assignments satisfying all clauses. This branching program has polynomial size because each

clause can be checked by a small subprogram and the subprograms can be connected sequentially.

Let B_0 be the constant-0 branching program. Then

$$\varphi \text{ is satisfiable} \Leftrightarrow B_\varphi \not\equiv B_0.$$

So

$$\varphi \in SAT \Leftrightarrow (B_\varphi, B_0) \in EQBP^c.$$

Thus $EQBP^c$ is NP-hard, which means EQBP is coNP-hard.

Therefore EQBP is coNP-complete.

6. Book 10.20

Question.

Show that

$$RP \cap coRP = ZPP.$$

We prove $RP \cap coRP = ZPP$.

$$RP \cap coRP \subseteq ZPP.$$

Let $A \in RP \cap coRP$. Then there are randomized polynomial-time machines:

- M_1 that never falsely accepts and accepts members with probability at least $1/2$,
- M_0 that never falsely accepts and accepts nonmembers with probability at least $1/2$.

Run one trial of each machine in parallel:

- if M_1 accepts, output accept;
- if M_0 accepts, output reject;
- otherwise output ? and repeat.

Each round has probability at least $1/2$ of terminating correctly, so the expected number of rounds is at most 2. The algorithm never outputs the wrong answer. Hence $A \in ZPP$.

$$ZPP \subseteq RP \cap coRP.$$

Let M be a ZPP machine for A . It never errs, and with probability at least $1/2$ it outputs the correct yes/no answer instead of ?.

To get an RP machine for A , run M once and treat ? as reject. Then:

- if $w \notin A$, M never outputs accept, so there are no false accepts;
- if $w \in A$, M accepts with probability at least $1/2$.

Thus $A \in RP$.

To get an RP machine for A^c , run M once and treat ? as reject but swap the roles of accept and reject. Therefore $A^c \in RP$, i.e. $A \in coRP$.

Hence $A \in RP \cap coRP$.

So $RP \cap coRP = ZPP$.

7. Book 10.19

Question.

Show that if $NP \subseteq BPP$, then $NP = RP$.

Assume $NP \subseteq BPP$. We prove $NP = RP$.

It suffices to show $SAT \in RP$, because SAT is NP-complete and $RP \subseteq NP$.

Let B be a BPP algorithm for SAT. Amplify B so that on formulas of size n , each query has error probability at most $1/(10n)$.

Now decide SAT by self-reduction:

1. Run B on φ . If it says unsatisfiable, reject.
2. Otherwise, for $i = 1, \dots, n$, run B on the two restricted formulas

$$\varphi[x_i = 0], \quad \varphi[x_i = 1].$$

Choose a value for x_i for which B says satisfiable. Continue recursively with that restriction.

3. At the end, we obtain a complete assignment a . Evaluate $\varphi(a)$ deterministically. Accept iff a satisfies φ .

This algorithm never falsely accepts: if φ is unsatisfiable, no assignment found at the end can satisfy it, so the final verification rejects.

If φ is satisfiable, then along some satisfying branch all the amplified BPP answers are correct with probability at least $9/10$, and in particular with probability greater than $1/2$.

On that event the algorithm reconstructs a genuine satisfying assignment and accepts.

Thus SAT is in RP. Therefore $NP \subseteq RP$. Since always $RP \subseteq NP$, we conclude

$$NP = RP.$$